Q. 1. Binary search to search for an element in a sorted array using: Divide and conquer approach and analyze time complexity.

Q. No.

- Introduction :-

It is divide and conquer based algorithm where the search domain is divided into half after each recursive call.

Binary search calculates the mid-point and compares with the key. Thus, reducing half of the search domain.

It uses recursive function.

- Objectives :-
  - To efficiently search for element in sorted array.
  - To implement Divide and conquer technique.

- Code :

```
Binary search (A, l, mid, r)
{
  if (l ≤ r)
  {
    mid = ⌈(l+r)/2⌉
    if (key > A[mid])
        Binarysearch (A, mid+1, mid, r);
    elseif (key < A[mid])
        Binarysearch (A, l, mid, mid-1);
    else (key == A[mid])
        printf ("search is found");
    }
    else
     printf ("the search is not found");
}
```

Algorithm :-

Step 1 : Start

Step 2 : Input the sorted array A of size n

Step 3 : Set left, right and mid

Step 4 : If left is ≤ right then

Calculate mid = $\lceil \frac{l+r}{2} \rceil$

if A[mid] = key, return the position of element

if key < A[mid], set right = mid - 1

otherwise left = mid + 1

Step 5 : if loop ends, element is not found.

Step 6 : End

Program :-

```
#include<stdio.h>
#include<stdlib.h>
int binarysearch(int A[], int l, int r, int key) // Binary search function
{
  if (l<=r)
  {
    int mid = (l+r)/2; // find middle index
    if (key > A[mid]) // search right half
      return binarysearch(A, mid+1, r, key);
    else if (key < A[mid]) // search left half
      return binarysearch(A, l, mid-1, key);
    else
      return mid; // element found
  }
  return -1; // element not found
}
int main()
{
  int num, key, index, i;
  printf("Enter the number of data:");
  scanf("%d", &num);
```

```
int A[num];
Printf("Enter the data in ascending order:\n");
for(i=0;i<num;i++)
{
  scanf("%d",&A[i]);
}
printf("Enter the number to be searched:");
scanf("%d",&key);
index = binarysearch(A,0,num-1,key);
if(index != -1)
    printf("search is found at index %d.\n",index);
else
    printf("search is not found.\n");
return 0;
}
```

Time complexity

Binary search repeatedly divides the sorted array into two halves.

Output:

```
Enter the number of data : 5
Enter the data in ascending order:
8
10
18
27
30
Enter the number to be searched: 27
search is found at index 3.
```

Time complexity

Binary search repeatedly divides the sorted array into two halves. The recurrance relation is:

$$T(n) = T(n/2) + 1$$

Complexities:

Best case : O(1)

Average case: O(log n)

Worst case : O(log n)

Conclusion:

Binary search is an efficient searching algorithm based on the Divide and conquer approach. It repeatedly divides the sorted array into halves and reduces the search space, resulting in a time complexity of $O(\log n)$. Therefore, it is suitable for searching elements in large sorted datasets.

CamScanner

b. Min-max algorithm to find minimum and maximum elements in a list.

Introduction:

The min-max algorithm is used to find the minimum & maximum elements in a list of numbers. Using Divide & conquer approach, this method reduces the number of comparisions compared to the normal linear approach.

Objective:

- To find minimum and maximum elements of an array.
- Use Divide & conquer technique.

Algorithm:

1. Start
2. Input the number of elements n
3. Input the array elements A [0.. n-1).
4. If the array has only one element, then it is both minimum and maximum.
5. If there are two elements, compare them and assign min and max.
6. Otherwise:
   - Divide the array into two halves.
   - Find the min and max of each half recursively.
   - Compare the results to get the final min & max.
7. Display the minimum & maximum elements.
8. Stop

Code:

```
# include <stdio.h>
// function to find minimum and maximum using divide & conquer
void minmax (int A [], int x+1, int r, int )

{ int mid;
  if (l==r) // if there is only one element

  { min = max = A [l];

  } else if [l==r-1 ] // Two elements
```

```
{
  if (A[l] < A[r])
  {
    min = A[l];
    max = A[r];
  }
  else
  {
    min = A[r];
    max = A[l];
  }
}
else
{ mid = (l+r)/2;
  minmax(A, l, mid);
  int min1 = min, max1 = max;
  minmax(A, mid+1, r);
  int min2 = min, max2 = max;
  if (min1 < min2)
     min = min1;
  else
     min = min2;
  if (max1 > max2)
     max = max1;
  else
     max = max2;
}
int main()
{ int n,i;
  printf("enter number of elements: ");
  scanf("%d", &n);
  int A[n];
  printf("enter elements:\n");
  for(i=0; i<n; i++)
     scanf("%d", &A[i]);
```

```
    minmax(A, 0, n-1);
    printf("minimum element = %d\n", min);
    printf("maximum element = %d\n", max);
    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter elements:
27
8
10
30
18
minimum element = 8
maximum element = 30
```

Best case: O(n), Average case: O(n), worst case: O(n)

Conclusion:

The min-max algorithm using Divide and conquer efficiently determines the smallest & largest elements in an array.

Q.2. Write a programs to sort a list of integers using merge sort algorithm. a)

a) Introduction:

merge sort is a sorting algorithm based on the Divide and conquer approach. It divides the array into smaller subarrays until each subarray contains a single element. Then the subarrays are merged in sorted order to produce the final sorted array. It is efficient & stable for large datasets.

Objective:

- To implement the merge sort algorithm.
- To use Divide and conquer Technique.

Algorithm:

1. Start
2. Input the nomber of element n.
3. Divide the array into two halves.
4. Recursively apply merge sort on both halves.
5. merge the two sorted halves into a singk sorted array.
6. Repeated until the array becomes completely sorted.
7. Stop

code:

```
void merge(int A[],int l,int m,int r)
{int i=l, j=m+1, K, B[100];
 for(K=l;K<=r;K++)
 {if(i>m)
     B[K]=A[j++];
  else else if(j>r)
     B[K]=A[i++];
  else if(A[i]<A[j])
     B[K]=A[i++];
  else
     B[K]=A[j++];
 }
```

```
    for(k=l ; k<=r, k++)
      A[k] = B[k];
}
void mergesort(int A[], int l, int r)
{
   if (l<r)
   { int m=(l+r)/2;
     mergesort (A,l,m);
     mergesort(A,m+1,r);
     merge(A,l,m,r);
   }
}
int main()
{ int n,i, A[100];
  printf("Enter number of elements:");
  scanf("%d",&n);
  printf("Enter elements :\n");
  for(i=0;i<n;i++)
    scanf("%d",&A[i]);
  mergesort(A,0,n-1);
   printf("Sorted elements :\n");
   for(i=0;i<n;i++)
      printf("%d ",A[i]);
   return 0;
}
```

Time complexity

Best case : $O(n(\log n))$

Average case : $O(n \log n)$

Worst case : $O(n \log n)$

Output:

```
Enter number of elements: 5
Enter elements :
27
8
10
30
18
```

Sorted elements :

8 10 12 27 30

Conclusion :

mergesort is an efficient sorting algorithm that uses the Divide and conquer technique. It consistently performs in O(n log n) time for all cases and is suitable for sorting large datasets.

CamScanner

b. Quick sort

Introduction:

Quick sort is a popular & efficient sorting algorithm based on the Divide & conquer method. It selects a pivot element, partitions the array into two parts & recursively sorts the subarrays.

Objectives:

- To implement the Quick sort algorithm using partitioning to sort list of integers.

Algorithm:

1. Start
2. Choose a pivot element from the array.
3. Partition the array so that:
   - Element smaller than pivot go to the left.
   - Element greater than pivot go to the right.
4. Recursively apply Quick sort on both partitions.
5. Stop

Code:

```
#include<stdio.h>
int partition(int A[], int low, int high)
{ int pivot = n[low];
  int i = low + 1, j = high, temp;
  while(i<=j)
  {
    while(A[i]<=pivot && i<high)i++;
    while(A[j]>pivot)j--;
    if(i<j)
    { temp = A[i];
      A[i] = A[j];
      A[j] = temp;
    }
  }
```

```
    A[low] = A[j];
    A[j] = pivot;
    return j;
}
void quicksort(int A[], int low, int high)
{
  if(low < high)
  {
    int p = partition(A, low, high);
    quicksort(A, low, p-1);
    quicksort(A, p+1, high);
  }
}
int main()
{
  int n, i, A[100];
  printf("Enter number of elements: ");
  scanf("%d", &n);
  printf("enter elements: \n");
  for(i=0; i<n; i++)
    scanf("%d", &A[i]);
  quicksort(A, 0, n-1);
  printf("Sorted elements: \n");
  for(i=0; i<n; i++)
    printf("%d", A[i]);
  return 0;
}
```

Output:

```
Enter number of elements: 6
Enter elements:
27
180
19
10
30
18
Sorted elements:
10 18 19 27 80 180
```

Time complexity:

Best case : $O(n \log n)$
Average case : $O(n \log n)$
Worst case : $O(n^2)$

Conclusion:

Quick sort is a fast sorting algorithm that uses partitioning and recursion. It performs very efficiently in average case but may degrade to $O(n^2)$ in the worst case when the pivot is poorly chosen.

CamScanner

Q.2 Randomized Quick sort

Introduction :-

Randomized Quick sort is an improved version of Quick sort where the pivot is chosen randomly instead of selecting a fixed element. This reduces the probability of encountering the worst case.

Objective:

- To implement Randomized Quick sort.

Algorithm:

1. Start
2. Choose a random pivot from the array.
3. Swap the pivot with the first element.
4. Partition the array based on the pivot.
5. Recursively apply Randomized Quick sort to both partitions.
6. Stop

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void swap(int *a, int *b)
{ int temp = *a;
  *a = *b;
  *b = temp;
}
int partition (int A[], int low, int high)
{ int pivot = A[low];
  int i = low+1 , j = high;
  while (i <= j)
  { while (A[i] <= pivot && i <= high) i++;
    while (A[j] > pivot) j--;
```

```
if (i<j)
  swap(&A[i],&A[j]);
  return j;
}
void randomized_quicksort(int A[],int low, int high)
{
  if (low<high)
  { int pivot = low + rand() % (high -low+1);
    swap (&A[low], &A[pivot]);
    int p= partition (A, low, high);
    randomized_quick(A, low, p-1);
    randomized_quick(A, p+1, high);
  }
}
int main()
{
  int n, i, A[100];
  printf("enter number of elements:");
  scanf("%d", &n);
  printf("enter elements:\n");
  for(i=0 ; i<n ; i++)
     printf("%d", A[i]);
  return 0;
}
```

Output:

```
enter number of elements: 5
enter elements:
 27
 8
 10
 18
 30
Sorted elements:
 8 10 18 27 30
```

Time complexity

Best case : $O(n \log n)$

Average case : $O(n \log n)$

Worst case : $O(n^2)$ (rare)

Conclusion:

Randomized Quick sort improves the standard Quicksort by choosing a random pivot, which reduces the chance of worst case performance.

CamScanner

Q. 3. Heap Sort

Introduction :-

A heap is a complete binary tree that satisfies the heap property. In a max heap, the parent node is always greater than its children.

Objective:

- To implement Heap sort using max heap

Algorithm:

Heapify

1. Assume node i is root.
2. Find left child $2i+1$ and right child $2i+2$.
3. compare root with children.
4. Swap with the largest child if needed.
5. Recursively apply heapify

- Build Heap

1. Start from the last non-leaf node.
2. Apply heapify to each node up to the root.

- Heap sort

1. Build a max heap.
2. Swap the first element with the last element.
3. Reduce heap size
4. Heapify the root.
5. Repeat until array is sorted

Code :-

```
#include <stdio.h>
void heapify (int A[], int n, int i) // function to maintain heap property
{
    int largest = i; // Assume root is largest
    int left = 2*i+1; // left child
```

```
    int right = 2*i+2; // Right child
    if (left < n && A[left] > A[largest]) // check if left child is larger
        largest = left;
    if (right < n && A[right] > A[largest] // check if right child is larger
        largest = right;
    if (largest != i) // if largest is not root, swap and heapify again
    {
        int *temp = A[i];
        A[i] = A[largest];
        heapify (A, n, largest); // Recursive heapify
    }
}
void heapsort(int A[], int n) // Heap sort function
{
    int i;
    for(i=n/2-1 ; i>=0 ; i--) // Build max heap.
        heapify(A,n,i);
    for(i=n-1 ; i>0; i--) // Extract elements from heap
    {
        int temp = A[0]; // swap first and last element
        A[0] = A[i];
        A[i] = temp;
        heapify (A,i, 0); // Heapify reduced heap
    }
}
int main()
{ int n, i, A[1000];
    printf("Enter number of elements : ");
    scanf("%d", &n);
    printf("Enter elements : \n");
    for(i=0; i<n ; i++) // input elements
        scanf("%d", &A[i]);
```

```
heapsort (A,n); // call heap sort
printf ("sorted elements: \n");
for (i=0; i<n ; i++) // Display sorted array
   printf ("%d", A[i]);
```

Output:

```
Enter number of elements : 5
Enter elements
 8
 12
 14
 20
 21
Sorted elements :
 8 12 14 20 21
```

Time complexity:

Best case : O(n log n)

Average case : O(n log n)

Worst case : O(n log n)

CamScanner

b. Find the k-th smallest element using order statistics, including Brute-force selection, expected linear Time selection and worst-case linear Time selection.

Introduction:-

The k-th smallest element problem is part of order statistics The brute-force method simply sorts the array and then selects the k-th element.

Randomized selection uses a random pivot similar to Quick sort to partition the array. It finds the k-th smallest element with expected linear time complexity.

worst case linear time selection (median of medians) guarantees linear time selection even in the worst case by carefully choosing a pivot.

Objective :-

- To find the k-th smallest element in an array using the brute-force approach.
- To implement an algorithm that finds the k-th smallest element using randomized selection.
- To determine the k-th smallest element in worst-case linear time.

Brute Force

Algorithm:

1. Start
2. Input array elements
3. Return the element at position k-1.
4. Stop

Code :-

```
#include <stdio.h>
int main()
{
  int n,j,k,temp,A[100];
  printf("enter number of elements:");
  scanf("%d",&n);
  printf("enter elements:\n");
  for(i=0;i<n,i++) // input array elements
    scanf("%d",&A[i]);
  for(i=0;i<n-1;i++) // Bubble sort to sort the array
    for(j=0;j<n-i-1;j++)
      if(A[j]>A[j+1])
      {
        temp=A[j];
        A[j]=A[j+1];
        A[j+1]=temp;
      }
    printf("enter k:");
    scanf("%d",&k);
    printf("k-th smallest element=%d",A[k-1]);
                // Display kth smallest element
    return 0;

}
```

Output:

```
enter number of element 5:
Enter elements:
 8
27
18
 7
30
enter k:2
k-th smallest element = 8
```

Time Complexity:

Best case : O(n log n)
Average case : O(n log n)
Worst case : O(n log n)

→ Expected Linear Time selection (Randomized selection)

Algorithm :-

1. Start
2. Choose a random pivot
3. Partition the array
4. If pivot position = k, return pivot
5. If pivot position > k, search left side
6. otherwise search right side
7. Stop

Code:

```
#include<stdio.h>
int partition(int A[], int l, int r) // partition function used
                                        in quick selection
{
  int pivot = A[r]; //choose last element as pivot
  int i = l-1, j, temp;
  for(j=l; j<r; j++)
  {
    if(A[j] <= pivot) //move smaller elements to left side
    {
      i++;
      temp = A[i];
      A[i] = A[j];
      A[j] = temp;
    }
  }
```

```
    temp = A[i+1]; //place pivot in correct position
    A[i+1] = A[r];
    A[r] = temp;
    return i+1;
}
// function to find Kth smallest element
int quickselect(int A[], int l, int r, int K)
{
    if (l<=r)
    {
    int p = partition(A,l,r); //partition array
    if (p==K) // if pivot is Kth element
      return A[p];
    else if (p>K) //search left side
       return quickselect(A, l, p-1, K);
    else  //search right side
       return quickselect(A,p+1, r, K);
    }
}
int main()
{
    int A[100], n, i, K;
    printf("enter number of elements: ");
    scanf("%d", &n);
    for(i=0; i<n; i++) //input array elements
       scanf("%d", &A[i]);
    printf("enter K:");
       scanf("%d", &K);
    // Display Kth smallest element
    printf("K-th smallest = %d", quickselect(A, 0, n-1, K-1));
    return 0;
    }
```

Output:

```
Enter number of elements: 5
3
18
5
27
2
Enter k: 3
Kth smallest element = 5
```

Time complexity:

Best case : O(n)
Average case : O(n)
Worst case : O(n²)

→ Worst-case Linear Time Selection (median of medians)

Algorithm:

1. Start
2. Divide array into group of 5 elements
3. Find median of each group.
4. Recursively find median of these medians
5. Use this median as pivot
6. Partition array
7. Recursively search required part.

Time complexity

Best case : O(n)
Average case : O(n)
Worst case : O(n)

CamScanner

Code :

```
#include <stdio.h>
int Kth smallest(int A[], int n, int k) // function to find Kth
                                          smallest element
{
  int i, j, temp;
  for(i=0; i<n-1; i++) // sort array using simple comparision sort
    for(j=i+1; j<n; j++)
      if(A[i]>A[j])
      {
        temp= A[i];
        A[i]= A[j];
        A[j]= temp;
      }
  return A[k-1]; // Return Kth smallest element
}
int main()
{ int n, i, k, A[100];
  printf("enter number of elements: ");
  scanf("%d", &n);
  for(i=0; i<n; i++) // Input elements
    scanf("%d", &A[i]);
  printf("enter K:");
  Scanf("%d", &k);
    // print Kth smallest element
  printf("K-th smallest element = %d", Kth smallest(A,n,k));
  return 0;
}
```

Output:

```
Enter number of elements : 5
90
4
13
6
27
Enter K:2
K-th Smallest element = 6
```

Conclusion:

The Kth smallest element problem can be solved using different order statistics methods. The Brute-force selection method is simple & works by sorting the array first, but it is less efficient for large datasets. The expected linear Time selection improves performance by using a random pivot and usually works in O(n) time on average. The worst case linear time selection guarantees O(n) time even in the worst case by carefully choosing the pivot, although. It is more complex to implement.

CamScanner